REACT · STATE YÖNETIMI

useReducer

Karmaşık state'i tek bir reducer ile yönetmek

Bugünkü yol haritası

01

Sorun

Dağınık setState çağrıları neden zorlaşır

02

useReducer ve reducer

state, dispatch, action ve saf fonksiyon

03

Payload ve sık yapılan hata

Action ile veri taşımak, state'i doğrudan değiştirmemek

04

useState ile karşılaştırma

Hangisi ne zaman, alıştırma ve tekrar

SORUN

State büyüdükçe, setState'ler dağılır.

Bir form düşünün: ad, e-posta, yükleme durumu, hata mesajı. Her biri ayrı useState ile tutulursa, bir olayda dört ayrı yerde güncelleme yapmak zorunda kalırsınız. Bir tane unutulursa, arayüz tutarsız hale gelir.

BÖLÜM 1 · USEREDUCER

useReducer nedir?

State'i güncelleme mantığını bir reducer fonksiyonunda toplayan React hook'u.

```
const
[state, dispatch] = useReducer(reducer, initialState);
```

state

Komponentin o anki verisi. Doğrudan değiştirilmez.

dispatch

Reducer'a "ne olduğunu" anlatan bir action gönderir.

reducer

(state, action) alır, yeni state'i döndürür.

Reducer: saf bir fonksiyon

```
function reducer(state, action) {
  switch (action.type) {
    case "increment": return { count: state.count + 1 };
    case "decrement": return { count: state.count - 1 };
    default: return state;
  }
}
```

Aynı girdi, aynı çıktı

State'i değiştirmez

Yan etki yapmaz

Tam örnek: sayaç

```
function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <p>Sayı: {state.count}</p>
      <button onClick={() => dispatch({ type: "increment" })}>+1</button>
      <button onClick={() => dispatch({ type: "decrement" })}>-1</button>
    </div>
  );
}
```

Butona tıklamak state'i doğrudan değiştirmez; yalnızca "ne olduğunu" dispatch eder.

Action ile veri taşımak

```
function todos(state, action) {
  switch (action.type) {
    case "added":
      return [...state, { id: Date.now(), text: action.payload }];
    case "removed":
      return state.filter((t) => t.id !== action.payload);
    default: return state;
  }
}
```

```
dispatch({ type: "added", payload: "Ders çalış" });
```

BÖLÜM 1 · SIK YAPILAN HATA

State’i doğrudan değiştirme

Yanlış

```
case "increment":
  state.count++;
  return state;
```

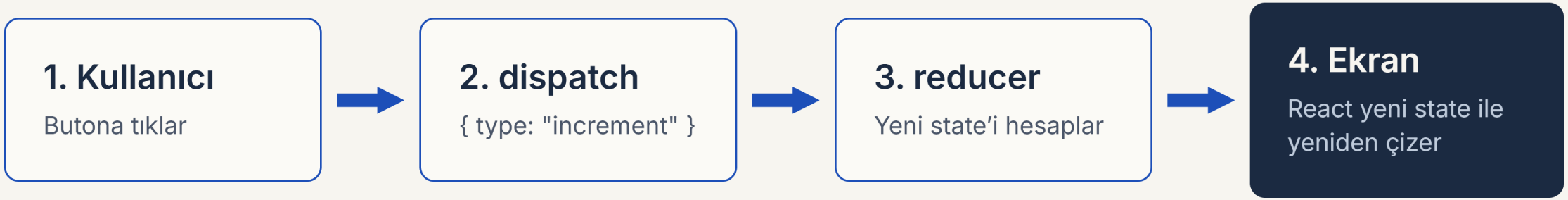
Aynı nesne döndüğü için React değişikliği fark etmez, ekran yenilenmez.

Doğru

```
case "increment":
  return {
    ...state,
    count: state.count + 1,
  };
```

Yeni bir nesne döner. Redux’ta da kural aynıdır.

Veri hangi yoldan akar?



Asıl iş reducer'da. Bileşen yalnızca ne olduğunu bildirir.

useState mi, useReducer mı?

Konu	useState	useReducer
Tek, basit değer	Uygun	Gereksiz karmaşık
Birbirine bağlı birkaç alan	Dağınık setState çağrıları	Tek dispatch ile hepsi güncellenir
Güncelleme mantığı	Event handler'ın içinde	Reducer içinde, tek yerde
Test edilebilirlik	Bileşene bağlı, zor	Reducer saf fonksiyon, kolay test

Kural: tek değer için useState. İlişkili, karmaşık state için useReducer.

ALIŞTIRMA 1 · USEREDUCER

Todo listesi yapalım

1. todos state’ini useReducer ile kurun.
2. added, toggled ve removed action’larını yazın.
3. Her todo için tamamla ve sil butonları ekleyin.

Süre: 20 dakika

Bonus: cleared ile tamamlananları temizle

HIZLI TEKRAR

Dört soru

1

Reducer neden saf bir fonksiyon olmak zorunda?

2

dispatch tam olarak ne yapar?

3

State'i doğrudan değiştirmek ekranı neden yenilemez?

4

Hangi durumda useState yerine useReducer seçersin?

SONRAKI DERS

Redux: aynı reducer, tüm uygulama için

Bugün bileşen içinde öğrendiğimiz dispatch, action ve reducer fikrini, bir sonraki derste tek bir store ile uygulama genelinde kullanacağız.